

Debugging Report

Issue 1: Layout Overflow (Analysis)

Detection Method:

We used the Flutter DevTools **Widget Inspector** to analyze the layout structure (as seen in the provided screenshot).

Description:

When inspecting the `Row` widget containing the input field and the "Add" button, we analyzed the risk of a `RenderFlex overflow error`. If the text input area attempts to take up infinite width (which is typical for a standard `TextField` inside a `Row` without constraints), it would crash the layout and display yellow-black caution tape on smaller screens.

Proposed Fix:

As verified in the Inspector screenshot, this issue is resolved/prevented by wrapping the `TextField` inside an `Expanded` widget. This constrains the text field to only occupy the remaining available width, ensuring the layout remains responsive and never overflows.

Issue 2: Web Performance Profiling

Detection Method:

We used the **Chrome Dev Tools Performance panel** (instead of Flutter DevTools) to capture a timeline trace while interacting with the Flutter Web app, as shown in the screenshot.

Description:

The recorded timeline reveals the breakdown of execution time on the web (e.g., 3,671ms for Scripting vs 79ms for Rendering). While no severe "red spike" jank is occurring in this simple app, Flutter Web heavily relies on JavaScript execution (Scripting) to manage the framework logic. If we were to load a massive list of tasks synchronously, the main thread would be blocked, causing dropped frames (jank).

Proposed Fix:

To keep the scripting time low and maintain smooth 60fps performance:

1. Always run performance profiling in `--profile` mode (not debug mode).
2. Continue using `ListView.builder` for the `task list` to lazily render only visible items, rather than loading all items into the DOM at once.
3. Utilize `const` constructors for static widgets (like `const Text('Taskly')`) to skip redundant rebuilds during the widget lifecycle.